tmforum

TM Forum Technical Report

Intent Specification

TR299

Maturity Level: **General Availability (GA)**	Team Approved Date: 04-Jul-2024
Release Status: Production	Approval Status: TM Forum Approved
Version 3.6.0	IPR Mode: RAND

©TM Forum 2024. All Rights Reserved.

Notice

Copyright © TM Forum 2024. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the TM FORUM IPR Policy, must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an “AS IS” basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

TM FORUM invites any TM FORUM Member or any other party that believes it has patent claims that would necessarily be infringed by implementations of this TM Forum Standards Final Deliverable, to notify the TM FORUM Team Administrator and provide an indication of its willingness to grant patent licenses to such patent claims in a manner consistent with the IPR Mode of the TM FORUM Collaboration Project Team that produced this deliverable.

The TM FORUM invites any party to contact the TM FORUM Team Administrator if it is aware of a claim of ownership of any patent claims that would necessarily be infringed by implementations of this TM FORUM Standards Final Deliverable by a patent holder that is not willing to provide a license to such patent claims in a manner consistent with the IPR Mode of the TM FORUM Collaboration Project Team that produced this TM FORUM Standards Final Deliverable. TM FORUM may include such claims on its website but disclaims any obligation to do so.

TM FORUM takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this TM FORUM Standards Final Deliverable or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Information on TM FORUM's procedures with respect to rights in any document or deliverable produced by a TM FORUM Collaboration Project Team can be found on the TM FORUM website. Copies of claims of rights made available for publication and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this TM FORUM Standards Final Deliverable, can be obtained from the TM FORUM Team Administrator. TM FORUM makes no representation that any information or list

of intellectual property rights will at any time be complete, or that any claims in such list are, in fact, Essential Claims.

Direct inquiries to the TM Forum office:

181 New Road, Suite 304
Parsippany, NJ 07054, USA
Tel No. +1 862 227 1648
TM Forum Web Page: www.tmforum.org

© TM Forum 2024. All Rights Reserved.

Table of Contents

Notice 2
Table of Contents 4
Executive Summary 7
Introduction 8
Scope 9
1. Notation and Dependencies 10
2. Intent Specification 12
2.1. Use cases for intent specifications 12
2.2. Intent manager interaction regarding intent specifications 13
3. Expressing Intent Specifications 14
3.1. Composite intent specifications 14
3.1.1. Mandatory intent specifications 14
3.1.2. Optional intent specifications 15
3.2. Intents expressions as building blocks 15
4. Templates 17
5. Object Templates 18
5.1. Explicit object assignment 18
5.2. Templates for a choice of objects and values 18
5.3. Dynamic set of allowed options 20
5.4. Available values determined by conditions 22
6. Content Templates 25
6.1. Content Options 25
6.2. Content Alternatives 27
6.3. Mandatory Content 29
7. Vocabulary Templates 32
7.1. Vocabulary constraints 32
7.2. Generic content from vocabulary templates 35
8. Combined use of templates 39
8.1. Example: Choice of metrics and required values. 39
9. Dependencies between choices 42
9.1. Conditions based on contextual information about the environment. 42
9.2. Conditions based on other content choices 43
9.3. Conditions based on chosen objects and values 43
9.4. Conditions based on used vocabulary 44
9.5. Conditionally applicable content 44

© TM Forum 2024. All Rights Reserved.

9.5.1. Mutual dependencies 46
9.6. Conditionally available object values 47
10. Generating end verifying intent expressions 49
11. Administrative Appendix 51
11.1. Document History 51
11.1.1. Version History 51
11.1.2. Release History 51
11.2. Acknowledgments 51
12. TR299 Intent Specification v3.6.0 - Appendix A Vocabulary 52
12.1. Appendix A: Vocabulary Reference 52
12.1.1. allowedValues 52
12.1.2. alternative 52
12.1.3. applicableIf 52
12.1.4. applicableIfNot 52
12.1.5. availableVocabulary 52
12.1.6. chosenAll 53
12.1.7. chosenAllFor 53
12.1.8. chosenAny 53
12.1.9. chosenAnyFor 53
12.1.10. chosenHandlingDomain 54
12.1.11. chosenIntentHandler 54
12.1.12. content 54
12.1.13. contentUsing 54
12.1.14. ContentTemplate 54
12.1.15. contentUsing 54
12.1.16. intentHandlerVendor 54
12.1.17. intentOwner 55
12.1.18. intentOwnerDomain 55
12.1.19. intentOwnerVendor 55
12.1.20. IntentSpecification 55
12.1.21. mandatory 55
12.1.22. mandatoryIntent 55
12.1.23. mandatorySpecification 55
12.1.24. ObjectTemplate 56
12.1.25. option 56
12.1.26. optionalIntent 56
12.1.27. optionalSpecification 56

© TM Forum 2024. All Rights Reserved.

12.1.28. selectValues 56
12.1.29. Template 57
12.1.30. templateType 57
12.1.31. usedVocabularyFor 57
12.1.32. valueSelected 57
12.1.33. valueSelectedFor 57
12.1.34. Vocabulary 58
12.1.35. VocabularyTemplate 58

© TM Forum 2024. All Rights Reserved.

Executive Summary

An intent specification defines rules and constraints regarding allowed content of intent. This allows generating and verifying well-formed intents. Many factors can be considered, such as the nature of an autonomous domain, the services and resources exposed by it and capabilities and properties of intent handlers. Intent specifications can be associated with product and service specifications within catalogs. Intent specifications are in this respect a new type of specification asset according to ODA/SID [sid]. This means they would determine how requirements regarding a product and service order can be expressed. Intent specifications are also used in intent manager capability profile to define the intent content an intent handler would support. This document defines the vocabulary to express intent specifications.

© TM Forum 2024. All Rights Reserved.

Introduction

Within the technical architecture of autonomous networks, intent managers use intent to exchange requirements. Intent managers in the role of intent owners are the source of intent. They would generate intent expression as needed to express the requirements to be communicated to another autonomous domain and the intent manager within. This intent manager receives intent and would coordinate the autonomous operation of the domain based on the requirements received within intent.

The autonomous domain that is receiving the intent might also expose products and services that can be ordered. In this context, intent would be used to specify the detailed requirements associated with an ordered product and service. This involves product and service specifications within a catalog. An individual product or service order would be expressed according to the product and service specifications. With the integration of intent within the APIs for service and product management an intent specification is needed to determine what an intent about a product or service can express or not. This document defines the content of an intent specification.

Another use case for intent specifications is the expression of intent manager capabilities. This includes a definition of the intent vocabulary and intent handler can understand and work with. An intent manager can use intent specifications to publish respective rules and constraints. This allows and intent owner to generate intent that is understood and likely to be accepted by an intent handler. At least it removes rejections for formal reasons, such as expressions being used that are not implemented and supported by the intent handler.

An intent specification consists of rules and constraints regarding the content of an intent expression. It allows specifying:

- Allowed vocabulary in the intent
- Allowed vocabulary in parts of the intent
- The structure of an intent
- Mandatory content
- Alternative content
- Optional content
- Allowed values and value ranges for metrics-based requirements
- Dependencies between choices

If an intent is generated according to these rules and constraints, it is a well-formed intent that can be understood by the receiving intent handler.

© TM Forum 2024. All Rights Reserved.

Scope

This document specifies the model for expressing intent specifications.

Revision Information

This revision of this document is v3.6.0. The version number indicates that it is aligned with and part of the version v3.6.0 of the TM Forum intent Ontology (TIO)

© TM Forum 2024. All Rights Reserved.

1. Notation and Dependencies

The Intent management elements model depends on the following models:

Model	Prefix	Namespace	Published by	Purpose in the model
Intent Specification	insp	`http://tio.models.tmforum.org/tio/v3.6.0/IntentSpecification/`	TM Forum	Refers to the model for expressing the content of intent specifications.
Intent Common Model	icm	`http://tio.models.tmforum.org/tio/v3.6.0/IntentCommonModel/`	TM Forum	General ontology model of intent and intent report expression. This document is part of the intent common model specification.
Intent Management Ontology	imo	`http://tio.models.tmforum.org/tio/v3.6.0/IntentManagmentOntology/`	TM Forum	Defines basic vocabulary and concepts of intent based operation. This document specifies vocabulary for intent management functions and their roles, as well as the types of intent models within the TM Forum Intent Ontology (TIO).
Conditions and Logical Operators Ontology	log	`http://tio.models.tmforum.org/tio/v3.6.0/LogicalOperators/`	TM Forum	Specifies logical operators to express logical relationships and the evaluation of truth values.
Quantity Ontology	quan	`http://tio.models.tmforum.org/tio/v3.6.0/QuantityOntology/`	TM Forum	Introduces quantities and quantity operators.
Set Operators	set	`http://tio.models.tmforum.org/tio/v3.6.0/SetOperators/`	TM Forum	Specification of set operators.
Function Definition Ontology	fun	`http://tio.models.tmforum.org/tio/v3.6.0/FunctionOntology/`	TM Forum	Basic expression of functions.
Metrics and Observations	met	`http://tio.models.tmforum.org/tio/v3.6.0/MetricsAndObservations/`	TM Forum	The metrics and observation model introduces metrics as measurable and observable items and defines vocabulary to manage observations about the metric

© TM Forum 2024. All Rights Reserved.

Model	Prefix	Namespace	Published by	Purpose in the model
Time Ontology in OWL	t	`http://www.w3.org/2006/time#`	W3C	Expression of date and time [owltime]
RDF version 1.1	`rdf`	`http://www.w3.org/1999/02/22-rdf-syntax-ns#`	W3C	Providing fundamental modeling basics [rdf11]
RDF Schema 1.1	`rdfs`	`http://www.w3.org/2000/01/rdf-schema#`	W3C	Providing schema for knowledge modeling [rdfs11]
XML Schema	`xsd`	`http://www.w3.org/2001/XMLSchema#`	W3C	Providing data types for literal objects [xsd-1] [xsd-2]
Examples	ex	http://www..example.org/	IANA	Reserved domain name for examples

Table 1.1: Model references

The Intent Specification model is based on the Resource Description Framework (RDF) [rdf, rdf_mt, rdf_primer] and the Resource Description Framework Schema (RDFS) [rdfs] published by the World Wide Web Consortium (W3C).

© TM Forum 2024. All Rights Reserved.

2. Intent Specification

Intent specifications are objects that contain sets of rules for how to express intent. An intent expression can be generated based on an intent specification by applying all its rules. This typically involves including statements prescribed by the intent specification in the intent expression is assembled. Furthermore, choices need to be made for optional and alternative content. The process that assembles and intent expression can therefore choose content as needed as long as its choice conforms with the intent specification.

Furthermore, intent specifications allow verification of intent expressions. An intent can be checked against an intent specification to determine if it is expressed according to all rules. If an intent expression conforms to an intent specification, this typically implies that the intent expression is allowed in its context of usage and understood by the receiver of the intent. This intent expression would be considered well-formed, containing sensible requirements and only uses vocabulary that is supported.

Intent is typically expressed based on a flexible federation of models. This includes the ontologies of RDF, RDFS and XML schema as foundation combined with models of the TM Forum intent ontology (TIO) and any additional custom extension. The TIO includes the intent base ontology, the intent common model and TM Forum defined intent extension models. Constituent models in this model federation add vocabulary that can be used within the intent expression.

The model for expressing intent specifications is designed work with any model included in a model federation for intent expression. There is no explicit relationship to vocabulary in intent expression models. This means, even new additions to the model federations can be considered in intent specifications without the need to update or adjust the intent specification model. For example, if an intent extension model defines a new specialization of expectations of context, the intent specification can express choices, rules and constraints concerning these additional elements.

2.1. Use cases for intent specifications

An intent specification can be used in many different contexts and use cases. For example:

Intent aware service and product management:

Intent specifications can reside in a product and service catalog and be associated with service specifications or product specifications. In this context, an intent specification would define, what intent content is allowed to be used for expressing service or product requirements for service or product instances according the associated service and product specification. In this use of intent specification it is possible to qualify a service against the service specification using the TMF 645 API and in the same qualification request present an intent and qualify it against an associated intent specification. This works similarly for product qualification in combination with intent qualification using the TMF 679 API. An intent specification associated with product and service management often sets boundaries for sensible requirements in terms of achievable value ranges and goals. It is therefore constraining what the source of the service or product order can require.

© TM Forum 2024. All Rights Reserved.

Intent Manager Capability:

Another use of intent specifications is within intent manager capability profiles [TR298]. With intent manager capability profiles an instance of an intent management function expresses what it is capable of and what models, interfaces, vocabulary, etc. it would support and understand. In this context an intent specification would be the set of rules that determine the intent structure and content supported by an intent manager. It prevents that intent sources express intent that would be rejected by an intent handler, because it does not support vocabulary and features used in the intent expression. For example optional intent extension models might not be implemented by an intent handler. The intent manager capability profile for this intent handler would use intent specifications that clearly state, what is supported and communicate this to any party that would generate intent to be sent to this intent handler.

2.2. Intent manager interaction regarding intent specifications

Intent specifications are primarily used by intent managers in the role of intent owners. They consult intent specifications when assembling intent to be sent to an intent handler. A typical use case would be intent about an ordered service. The intent owner and the source of the service owner are the same entity. This intent owner has selected a service to be ordered. It would retrieve the intent specification that is associated with this service from the service catalog. The intent owner would then express the intent in a way that it conforms with the intent specification. It needs to respect the constraints set by the intent specification to avoid rejection by the intent handler.

Intent handlers can evaluate if the intent follows applicable intent specifications when assessing its acceptance. Intent handlers can therefore reject intent that does not conform with an intent specification and state this as rejection reason. Similar rejections can happen at every intent update.

An intent specification can contain conditions. They are evaluated when assembling an intent expression to be used for sending intent to an intent handler. The intent owner needs to make sure that the intent conforms to a respective intent specification including all conditions when it is sending the intent to an intent handler. As the evaluation result for conditions in intent specifications can change over time, available and allowed intent expression details are dynamic. Choices can become available or vanish over time. This means, it is possible that and intent becomes non-conforming to the applicable intent specification after the intent handler has accepted it and while it is handled. Intent handlers have a variety of options how to deal with this situation. They can silently ignore the mismatch with the specification and continue operating with it. This is the preferred way of handling this situation as it follows the principle that intent handlers cannot unilaterally change or remove intent after it was accepted and without the intent owner's instruction. Intent handlers can however send warnings about this to the intent owner within intent reports or even declare the intent degraded on the base that it is not conforming with the intent specification anymore. The correct intent handler reaction depends on the use case.

© TM Forum 2024. All Rights Reserved.

3. Expressing Intent Specifications

Intent specifications are instances of class `insp:IntentSpecification`. An intent is assembled in accordance to an intent specification by finding an intent expression that complies to all rules and constraints expressed by the intent specification. Details of how to express these rules and constraints are specified in the following chapters of this document.

3.1. Composite intent specifications

Intent specifications can be defined using other intent specifications. This creates a composite intent specification, where constituent intent specifications determine mandatory or alternative sets of rules and constraints. This means an intent conforms to an intent specification if it conforms to any, all, or a subset of constituent intent specifications. The detailed conformance logic is determined by the used properties that associate an intent specification to constituent intent specifications.

3.1.1. Mandatory intent specifications

The property `insp:mandatorySpecification` expresses that an eligible intent expression must conform to the intent specification provided in its range. This expresses, that the rules and constraints of that intent specification are mandatory for follow. Multiple statements using `insp:mandatorySpecification` with the same subject are in a conjunction (AND). This means an assembled intent expression must conform to all mandatory intent specifications.

For example:

```
ex:IS1
  a insp:IntentSpecification ;
.
ex:IS2
  a insp:IntentSpecification ;
.
ex:IS3
  a insp:IntentSpecification ;
  insp:mandatorySpecification ex:IS1, ex:IS2
.
```

This example shows the structure of an intent specification using two nested constituent intent specifications. For an intent to conform to the intent specification `ex:IS3` it is mandatory that it conforms to the intent specifications `ex:IS1` and `ex:IS2`. Note, that this example only shows the structure of nested intent specifications, but it does not contain any more detailed rules and constraints regarding the content of intent that is assembled accordingly.

© TM Forum 2024. All Rights Reserved.

3.1.2. Optional intent specifications

The property `insp:optionalSpecification` expresses that an intent expression can be assembled following the intent specifications provided in its range. Multiple statements using `insp:optionalSpecification` with the same subject are in a disjunction (OR). This means an intent expression conforms overall if it conforms to at least one and potentially several of the optional specifications.

If an intent specification uses a mix of mandatory and optional constituent intent specifications, conforming to mandatory ones is sufficient.

For example:

```
ex:IS1
  a insp:IntentSpecification ;
.
ex:IS2
  a insp:IntentSpecification ;
.
ex:IS3
  a insp:IntentSpecification ;
  insp:optionalSpecification ex:IS1, ex:IS2
.
```

This example shows the structure of an intent specification using two optional constituent intent specifications. For an intent to conform to the intent specification `ex:IS3` it must conform to at least one of the intent specifications `ex:IS1` and `ex:IS2`. Note, that this example only shows the structure of nested intent specifications, but it does not contain any more detailed rules and constraints regarding the content of intent that is assembled accordingly.

3.2. Intents expressions as building blocks

Intent specifications use intents expressed according to the TM Forum intent ontology (TIO) to prescribe content of intent expressions. This means intent specifications can specify intents and intent elements directly. This would prescribe to use exactly this intent or exactly an intent element as expressed.

It is not strictly required that the exact intent instance of an intent or intent element is used as specified in the intent specification. A conforming intent can also use distinct instances for the intent object and all intent elements, as long as every element of the intent provided and all properties to associate the intent elements with each other are matched. This means the conforming intent has the same content and structure and only differs in the identifiers of the used individuals.

The property `insp:optionalIntent` assigns intent objects to an intent specification. An intent expression that resembles any of these intents conforms to the intent specification.

The property `insp:mandatoryIntent` intent assigns intent expressions that must be used. This means an intent expression conforming to the intent specification must conform to this intent expression including all rules and dynamic content choices it allows.

© TM Forum 2024. All Rights Reserved.

This way of providing intent specifications is very restrictive as it leaves little room for variation and choice when a conforming intent is assembled according to the intent specification. More flexibility and choice is introduced in chapter 4 and following chapters of this document.

For example:

```
ex:IS1
  a insp:IntentSpecification ;
  insp:optionalIntent ex:I1, ex:I2 ;
  insp:mandatoryIntent ex:I3 , ex:I4
.
ex:I1
  a icm:Intent
.
ex:I2
  a icm:Intent
.
ex:I3
  a icm:Intent
.
ex:I4
  a icm:Intent
.
```

This example specifies that an intent expression conforming to intent specification ex:IS1 can be assembled based on `ex:I1` or `ex:I2`. In addition, a conforming intent must be assembled according to `ex:I3` and `ex:I4`. This means, when generating a resulting intent expression there is a choice to follow any of the two intents `ex:I1` or `ex:I2`. Whatever choice was made, resulting intent expression must also follow both mandatory intents `ex:I3` and `ex:I4`.

© TM Forum 2024. All Rights Reserved.

4. Templates

Templates are placeholders for modelling elements within an intent expression. This includes intents and intent elements such as targets, expectations, conditions or context. Templates can also be placeholders for data according to their datatype and distinct elements of RDF, RDFS or other foundational models. For example, there can be templates for containers and lists. A template would define the rules for how to express the model element it represents, including properties it can have or must have. This can also include allowed options, alternatives, or, if metrics are involved, allowed values and value ranges. A conforming intent would contain an element represented by the template for every individual template of the intent specification. The respective element would be constructed and expressed by following the template rules regarding statements, content, objects and values, etc., and it would be allocated in an intent structure according to the template's position in the intent specification. This makes templates a very flexible tool to describe construction rules for conforming intents.

Choices must satisfy all constraints set by templates to be considered valid content of a conforming intent. For example, if chosen content of an intent expression is allowed according to one template, but not allowed according to the rules of another template, it is not an eligible choice and cannot be used in the intent expression.

Templates were already used and shown in chapter 3. The range of properties, such as `insp:option`, `insp:alternative` or `insp:mandatory` is a template. These properties have in common that they specify that statements from the object in range can or must be taken into the subject. As the object is a template, this applies to statements chosen according to the template rules. However, templates can be used with a great variety of properties and whenever the intent specification would like to state rules for how to express and element of the intent.

This chapter focuses on the definition of templates and use cases that are based on using them within intent specifications.

Templates are instances of subclasses of class `insp:Template`. There are three subclasses of templates, object templates, content templates and vocabulary templates. Their definition and usage is specified in the following chapters 4, 5 and 6.

Applying all rules and constraints according to templates does not necessarily lead to an intent with sensible content that states achievable requirements. They rather ensure that the intent is well-formed and can be understood by an intent handler.

© TM Forum 2024. All Rights Reserved.

5. Object Templates

Templates which represent an object or a value within statements of the intent expression are referred to as object templates. An object template is an instance of class `insp:ObjectTemplate`, which is a subclass of `insp:Template`. Within intent specifications an object template can be used as substitute for any object within an intent expression. The property `insp:templateType` specifies what type of object an object template instance is representing and substituting in an intent expression that conforms to the intent specification. This also means the respective object template instance can be used at any place in the intent specification where an object of this class is expected and allowed.

An object template can represent any RDF resource. This includes instances of classes, properties and literals. Examples in the context of the TM Forum Intent Ontology are expectations, conditions, targets, metrics, quantities and contexts. It can also represent values of literals or objects defined by RDF and RDFS such as lists and containers.

Object templates have properties that define rules for objects that can be used in an intent in place of the template. An intent that conforms with the intent specification would be expressed by substituting the object template according to its properties. Typically, the template defines a choice of objects or values to be used in the intent expression.

The following chapters specify these properties and demonstrates their use in a variety of use cases.

5.1. Explicit object assignment

Object templates can use the property rdf:value to assign an object to the template. This is the object the template would represent and be substituted with in the expression of a conforming intent.

Functions according to the function definition ontology [TR292] provide a result value for their subject. This is equivalent to explicitly setting the result using rdf:value. Functions can therefore determine the object to be represented by the template.

5.2. Templates for a choice of objects and values

A conforming intent can be constructed by substituting the template with an object chosen from a set of possible candidates. The property `insp:allowedValues` is used to specify the set of candidate objects or values to choose from for substituting the object template in its domain within a conforming intent expression. The range of the `insp:allowedValues` property is `rdfs:Container`. Members of this container are the candidate objects to choose from when substituting the template for expressing the respective intent element. Consequently, the member objects would match the template type specified by the property `insp:templateType`. For example, if the candidate objects are quantity values, the template type would be the quantity class quan:Quantity or the quantity datatype:quan. Both are equivalent.

© TM Forum 2024. All Rights Reserved.

If an intent is assembled from an intent specification, each individual object template represents a distinct choice. This means a choice regarding an object template instance is substituted into the assembled intent at every reference to that object template instance. This enforces that the choice regarding an object template is made to satisfy all references to it. Consequently, if different choices shall be allowed from the same set of allowed values, multiple individual object templates need to be used, referring to the same container of values.

For example:

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1
  a icm:Target ;
  rdf:member ex:TT1
.
ex:T2
  a icm:Target ;
  rdf:member ex:TT1 ;
  rdf:member ex:TT2
.
ex:TT1
  a insp:ObjectTemplate ;
  insp:templateType ex:Slice ;
  insp:allowedValues ex:V1 ;
.
ex:TT2
  a insp:ObjectTemplate ;
  insp:templateType ex:Slice ;
  insp:allowedValues ex:V1 ;
  insp:allowedValues [ rdfs:member ex:SLice3, ex:Slice4 ]
.
ex:V1
  a rdfs:Container ;
  rdfs:member ex:SLice1, ex:Slice2
.
ex:E1
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice
.
```

This example shows the use of templates to define allowed choices for targets. The target `ex:T1` has one member, which is determined by the object `template ex:TT1`. It has allowed values `ex:Slice1` and `ex:Slice2` according to the container `ex:V1`. This means if an intent is expressed according to this intent specification the member for

© TM Forum 2024. All Rights Reserved.

this target can be either of these two slice instances. The template type of `ex:TT1` is `ex:Slice`. This is the class of the slice instances available to choose from.

The example intent specification also defines target `ex:T2`. This target has two members. The first one is chosen according to object template `ex:TT1`. This means, whatever is chosen for this object template must be used as member in both targets within a conforming intent expression. In addition, the target `ex:T2` has a second member with values chosen according to object template `ex:TT2`. As it is a distinct object template instance, it represents an independent choice. This object template has also allowed values according to `ex:V1`. In addition, it allows using `ex:Slice3` and `ex:Slice4`. This means the choice for object template `ex:TT2`, which is the choice for the second member of target `ex:T2` can be made from all four slice instances.

The following intent would be conforming with the intent specification discussed in this example :

```
ex:I2
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T3
  a icm:Target ;
  rdf:member ex:Slice1
.
ex:T4
  a icm:Target ;
  rdf:member ex:Slice1, ex:Slice4
.
ex:E2
  a icm:DeliveryExpectation ;
  icm:target ex:T2 ;
  icm:deliveryType ex:Slice
.
```

The target `ex:T3` was expressed according to `ex:T1` in the intent specification. It has one member `ex:Slice1`. This was one of the allowed values according to the `ex:V1` used in object template `ex:TT1`.

The target `ex:T4` was expressed according to `ex:T2` in the intent specification. The first member of target `ex:T4` must also be `ex:Slice1` as it is derived from the same choice made for object template `ex:TT1`. In addition, `ex:Slice4` was chosen for the second member of target `ex:T4`. It was chosen according to `ex:TT2`.

5.3. Dynamic set of allowed options

The range of the property insp:allowedValues is a container. Assigning members individually to this set as shown in chapter 4.4 is static, while members might need to be connected to dynamically changing pools of objects. A good example are network resources such as slice instances. New instances might be added or removed dynamically, and an intent should allow making requirements for any instance of a particular type of resources. This can be expressed using functions that provide container type result values. They allow to abstractly define and filter containers by describing properties of eligible members. Functions for constructing containers and

© TM Forum 2024. All Rights Reserved.

filtering their members are provided in the set operators model [TR292F]. Further functions and modeling approaches regarding containers from other models can also be used as needed.

For example:

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1
  a icm:Target ;
  rdf:member ex:TT1
.
ex:TT1
  a insp:ObjectTemplate ;
  insp:templateType ex:Slice ;
  insp:allowedValues [ set:intersection ( set:resourcesOfType ( ex:Slice )
                                          set:resourcesWithPropertyObject ( ex:region
ex:Subnet1 )
                                        )
                     ]
.
ex:E1
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice
.
```

This example shows the use of set functions to define allowed values or objects to be used as target members. In this example, an allowed member is of type `ex:Slice` and has the property `ex:region` with object `ex:Subnet1`. This is expressed by abstract containers. One represents the set of all resources of type `ex:Slice` and the other container represents the set of all resources in the region `ex:Subnet1`. The set of eligible options represented by the template `ex:TT1` is the intersection of the two containers. Resources might be defined in the knowledge base as:

```
ex:Slice1
  a ex:Slice ;
  ex:region ex:Subnet1 ;
.
ex:Slice2
  a ex:Slice ;
  ex:region ex:Subnet1 ;
.
ex:Slice3
  a ex:Slice ;
  ex:region ex:Subnet2 ;
```

© TM Forum 2024. All Rights Reserved.

```
.
ex:Node1
  a ex:GNodeB ;
  ex:region ex:Subnet1 ;
.
```

With these resources the individuals `ex:Slice1` and `ex:Slice2` would be in both constituent sets and therefore in the intersection set. These are therefore the eligible choices. The object `ex:Slice3` has the wrong region and `ex:Node1` has the wrong type. They are in one of the sets, but therefore not in the intersection. Consequently, the following intent conforms with the intent specification:

```
ex:I2
  a icm:Intent ;
  allOf ( ex:E1 )
.
ex:T2
  a icm:Target ;
  rdf:member ex:Slice1
.
ex:E2
  a icm:DeliveryExpectation ;
  icm:target ex:T2 ;
  icm:deliveryType ex:Slice
.
```

It substitutes the template with `ex:Slice1`, which is a member object of the allowed values container.

If later further resource instances are added to the network, which are also slices in region ex:Subnet1, they meet all the set construction and filtering criteria. Therefore, they would implicitly become eligible choices as well.

5.4. Available values determined by conditions

The main purpose of object templates is to set rules for eligible objects and values allowed to be used in intent expressions. This can be achieved by using conditions about the object and value.

```
<rdfs:Container> = insp:selectValues ( <rdfs:Resource> <log:Condition>
... )
```

The property `insp:allowedValues` uses a container to provide choices. The members of this container can be determined by the abstract function `insp:selectValues`. The first argument of `insp:selectValues` is a variable that represents the values to be chosen. The second and further arguments are conditions that use the variable in its first argument to express condition statements. Every value possible for the variable in the first argument according to the conditions is a member of the resulting container. The conditions are in a conjunction. This means, a value needs to satisfy all conditions to be included in the result container of an `insp:selectValues` function statement.

© TM Forum 2024. All Rights Reserved.

If multiple `insp:selectValues` statements are used for a node, their combined contribution is a union of individual results. This means, this node is a container with a superset of members across all result containers from individual `insp:selectValues` statements. If other combinations of individual results are needed, the container functions from the set operators ontology [TR292F] can be used.

Using the container determined by `insp:selectValues` statements as object of `insp:allowedValues` means that allowed value available to be chosen are the values that satisfy conditions of `insp:selectValues` statements.

For example: Using conditions to specify allowed value ranges.

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1
  a icm:Target ;
  rdf:member ex:Slice1
.
ex:E1
  a icm:PropertyExpectation ;
  icm:target ex:T1 ;
  log:allOf ( ex:C1 )
.
ex:C1
  a log:Condition ;
  quan:smaller ( [ met:lastValue ( ex:latency ) ]
                 ex:OT1
               )
.
ex:OT1
  a insp:ObjectTemplate ;
  insp:templateType quan:Quantity ;
  insp:allowedValues [ insp:selectValues ( _:value ex:C2 ... ) ]
.
ex:C2
  a log:Condition ;
  quan:inRange ( _:value
                 "10ms"^^quan:quantity
                 "100ms"^^quan:quantity
               )
.
```

© TM Forum 2024. All Rights Reserved.

This example shows an intent specification that uses intent `ex:I1` as possible intent. Within this intent, there is a property expectation using the condition `ex:C1` to express a requirement regarding latency. A possible value that can be required is determined by the object template `ex:OT1`. This object template expresses a condition for eligible values using the condition `ex:C2`. It also declares that the B-node `_:value` shall represent the template value in the condition statements. Here the condition is that the value from the value range of `10ms` to `100ms`. This means any value that meets this condition can be used in the intent expression substituting the object template. For example if the intent expresses a requirement for latency using "`20ms`" this conforms to the intent specification. Using a value of "`500ms`" does not meet the template condition and would not result in a conforming intent.

© TM Forum 2024. All Rights Reserved.

6. Content Templates

The intent specification model provides rules for conforming intent. It might need to specify that certain expressions and statements are mandatory and need to be included in a conforming intent. It should also be possible to specify that certain expressions and statements are allowed, but optional. And there are cases in which an expression or statement misty be included, but there are multiple eligible alternatives to choose from. The intent specification model introduces content templates for these use cases.

Content templates are instances of class `insp:ContentTemplate`, which is a subclass of `insp:Template`. The following subchapters specify how content templates are used. The common concept is that content templates contain statements that would be transferred into the intent expression of conforming intents. If this is optional or mandatory or if a choice of several alternatives is provided depends on the property used to anchor a content template within an intent. The following subchapters introduce these properties and explain their meaning and use cases.

For content templates, every property that assigns a content template to an intent element within an intent specification represents an individual choice. This means, that content templates instances can be re-used within multiple statements without automatically implying a dependency between choices. Dependencies between choices are discussed in chapter 7.

The property `insp:content` assigns the statements represented by the content template. The node used in its range is the source of statements to be transferred into the intent expression. All statements in that node shall be transferred. If `insp:content` is used multiple times within a content template, their contributions are additive and the superset of statements is applied.

The property `insp:contentUsing` assigns content chosen according to a vocabulary template. This is described in chapter 6.2.

A combined use of content templates and object templates is possible and provides flexible expressiveness in intent specification. This is discussed with examples in Chapter 5.4.

6.1. Content Options

Optional content refers to statements or sets of statements that can, but do not need to be included into a conforming intent.

The property `insp:option` specifies a set of statements that can be used in an intent expression. The range of `insp:option` is a template of class `insp:ContentTemplate`. The statements and expressions of this content template would substitute the insp:option statement within a conforming intent expression if this option is chosen.

If the object of an `insp:option` statement is an object template, the template rules determine valid statements that would substitute the `insp:option` statement.

For example:

```
ex:E1
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  insp:option ex:CT1 ;
```

© TM Forum 2024. All Rights Reserved.

```
  insp:option [ insp:content [ icm:deliveryType ex:Slice ] ]
.
ex:CT1
  a insp:ContentTemplate ;
  insp:content [ icm:deliveryDescription ex:SliceSpecification ;
                 icm:deliveryType ex:Slice ]
.
```

This example defines options for expression of a delivery expectation. It states that the delivery expectation can contain a statement using the property `icm:deliveryDescription` with object `ex:SliceSpecification` and `icm:deliveryType` with object `ex:Slice`. This option is described by the content template ex:CT1 through its `insp:content` property. The second option states that it is also possible to use the property `icm:deliveryType` with object `ex:Slice`. For this option the content template is provided inline as B-node. According to this specification the delivery expectation can contain any combination of these options including none of them.

The following example expectations demonstrate what would be conforming to the intent specification and what would not:

```
ex:E2
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpecification ;
  icm:deliveryType ex:Slice
.
ex:E3
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice
.
ex:E4
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
.
ex:E5
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpecification ;
.
ex:E6
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpecification ;
  icm:deliveryType ex:GNodeB ;
.
ex:E7
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpecification ;
```

© TM Forum 2024. All Rights Reserved.

```
  ex:someOtherProperty ex:Slice ;
.
```

The expectation `ex:E2` conforms, because it contains both statements from the first option.

The expectation `ex:E3` conforms, because it is using one of the options.

The expectation `ex:E4` conforms, because using none of the options is allowed.

The expectation `ex:E6` does not conform, because it only contains one of the two statements of the first option.

The expectation `ex:E6` does not conform, because it is using a statement that does not match any of the options. While it is using the same property as stated in the options, the object of the property is not covered by any of the options.

The expectation `ex:E7` does not conform, because it uses a statement with a property that is not covered by any option.

6.2. Content Alternatives

Alternative content refers to multiple alternative statements or sets of statements, of which exactly one must be included into a conforming intent expression.

The property `insp:alternative` specifies alternative content within intent specifications. Its range contains a list of all content templates representing viable alternatives. Each content template in the list alternative represents a statement or set of statements. Exactly one of these content templates and its statements needs to be used in place of `insp:alternative` when expressing an intent. This means a conforming intent expression must include a set of statements for every `inst:alternative` statement in the intent specification.

For example:

```
ex:E1
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  insp:alternative ( [ insp:content [ icm:deliveryDescription ex:SliceSpec1 ;
                                      icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
                                    ]
                     ]
                     [ insp:content [ icm:deliveryDescription ex:SliceSpec2 ;
                                      icm:chooseFrom [ rdfs:member ex:SliceC ex:SliceD ]
                                    ]
                     ]
                   )
.
```

© TM Forum 2024. All Rights Reserved.

This example defines alternative content for a delivery expectation. The first alternative contains a statement using `icm:deliveryDescription` with object `ex:SliceSpec1`. It also contains a statement with property `icm:chooseFrom` and a container with two slice instances as object. The second alternative is similar, but it uses `ex:SliceSpec2` as object for `icm:deliveryDescription` and a container with two different slice instances.

The following example expectations demonstrate what would be conforming to the intent specification and what would not:

```
ex:E2
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpec1 ;
  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
.
ex:E3
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice1 ;
  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
.
ex:E4
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
.
ex:E5
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpec2 ;
  icm:chooseFrom [ rdfs:member ex:SliceC ]
.
ex:E6
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpec2 ;
  icm:chooseFrom [ rdfs:member ex:SliceC, ex:SliceD, ex:SliceE ]
.
ex:E7
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryDescription ex:SliceSpec1 ;
  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
  icm:deliveryDescription ex:SliceSpec2 ;
  icm:chooseFrom [ rdfs:member ex:SliceC, ex:SliceD ]
.
```

© TM Forum 2024. All Rights Reserved.

The expectation `ex:E2` conforms, because it is using the exact statements of the first alternative.

The expectation `ex:E3` does not conform, because it is using none of the alternatives. Instead, it is using the property `icm:deliveryType` in combination with `icm:chooseFrom`. This is not covered by the intent specification.

The expectation `ex:E4` does not conform, because it is not using any of the alternatives and there are also no additional options. The property `insp:alternative` demands that one of its alternatives is chosen.

The expectation `ex:E5` does not conform, because the container used with the property `icm:chooseFrom` contains only `ex:SliceC`. While this included in the content template of one possible alternative, it was only partly used in `ex:E5`. This expression is therefore incomplete.

The expectation ex:E6 does not conform, because the container used with the property `icm:chooseFrom` contains only `ex:SliceE`. No alternative allows this.

The expectation `ex:E7` does not conform, because it is using the content from both alternatives although the property `insp:alternative` demands that only on alternative is applied.

6.3. Mandatory Content

Mandatory content refers to statements or sets of statements that must be included in the intent expression.

The property `insp:mandatory` specifies a content template that represents a statement or sets of statements that must be included in an intent expression. Its range contains a content template instance of class `insp:ContentTemplate`. This content template determines the statements that must be used as substitute for the `insp:mandatory` statement in a conforming intent expression.

According to the principles of intent specifications, statements that must be present in the intent expression can also be directly added. However, using the property `insp:mandatory` allows the use of templates. This enables dynamic construction of the statements to be used. It also provides an anchor point for conditions as described in chapter 5.5.

For example:

```
ex:E1
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice ;
  insp:mandatory [ insp:content [ icm:deliveryDescription ex:SliceSpec1 ;
                                  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
                 ]
.
```

This example defines several mandatory statements to be included in a delivery expectation. The property `icm:deliveryType` with object `ex:Slice` must be used also in the delivery expectation within a conforming intent expression. It is explicitly provided in the intent specification and therefore always included in the respective intent. The

© TM Forum 2024. All Rights Reserved.

example also shows explicit declaration of mandatory content according to a content template. The content template is assigned using the property insp:mandatory, thus declaring that its content must be used. The content template contains a statement with property `icm:deliveryDescription` and one with property `icm:chooseFrom`. Both need to appear in a conforming intent with their respective objects.

The following example expectations demonstrate what would be conforming to the intent specification and what would not:

```
ex:E2
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice ;
  icm:deliveryDescription ex:SliceSpec1 ;
  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
.
ex:E3
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice ;
  icm:deliveryDescription ex:SliceSpec1 ;
  icm:chooseFrom [ rdfs:Container ;
                   rdfs:member ex:SliceA ;
                   rdfs:member ex:SliceB ]
.
ex:E4
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice1 ;
  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ]
.
ex:E5
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice ;
  icm:deliveryDescription ex:SliceSpec1 ;
  icm:chooseFrom [ rdfs:member ex:SliceA, ex:SliceB ] ;
  ex:myProperty ex:Slice
.
ex:E6
  a icm:DeliveryExpectation ;
  icm:target ex:T1 ;
  icm:deliveryType ex:Slice ;
  icm:deliveryDescription ex:SliceSpec2 ;
  icm:chooseFrom [ rdfs:Container ;
                   rdfs:member ex:SliceC, ex:SliceD ] ;
.
```

Both expectation instances `ex:E2` and `ex:E3` conform, because they are using all mandatory statements exactly as expressed in the intent specification. Both are syntactically different, but describe the same graph.

© TM Forum 2024. All Rights Reserved.

The expectation `ex:E4` does not conform, because it is not using all mandatory statements. It is missing the statement with property `icm:deliveryDescription` from the content template used with `insp:mandatory`.

The expectation `ex:E5` does not conform, because in addition to mandatory statements it uses an additional statement with property `ex:myProperty`. This is not covered by the intent specification.

The expectation `ex:E6` does not conform. It is using statements with the right properties, but using objects such as `ex:SliceSpec2` or different members in set to choose from, which are not covered by the specification.

© TM Forum 2024. All Rights Reserved.

7. Vocabulary Templates

Vocabulary templates define sets of intent expression vocabulary that can be used in a conforming intent expression or a part of the intent. Vocabulary refers to classes, properties and instances defined by a model. This can be the vocabulary defined by constituent models of the TM Forum intent ontology (TIO) including the base ontology model, the intent common model and any intent extension models. It can also include any models outside the TIO if needed. For example, models within the TIO might depend on these models or use them to gain more expressiveness. Examples are RDF, RDFS and XML Schema.

A vocabulary template is an instance of class `insp:VocabularyTemplate`, which is a subclass of `insp:Template`. A vocabulary template is also a subclass of `imo:Vocabulary`, which is a subclass of `rdfs:Container`. Consequently, a vocabulary template expresses a set and the property `rdfs:member` is available to assign member objects. Furthermore, any function or operator for constructing and filtering containers is available to specify sets of members to be used for vocabulary templates.

The specification of vocabulary can use pre-defined sets of vocabulary that might be provided by individual models within the TIO. For example, the intent common model [TR290] defines the vocabulary container `icm:Vocabulary` enumerating all vocabulary of the model.

7.1. Vocabulary constraints

The property `insp:availableVocabulary` assigns a vocabulary template to an intent within an intent specification. It implies that the intent and all its elements can only be expressed using the vocabulary specified by the vocabulary template. Multiple `insp:availableVocabulary` assigned to the same subject are interpreted as union of the individual vocabulary templates. This means that vocabulary from any of the involved vocabulary templates is applicable.

The use of `insp:availableVocabulary` in an intent specification introduces a constraint for intent expression. Only the specified vocabulary can be chosen to be used. This has effects on other templates. For example, if the content specified by a content template for an intent element uses vocabulary outside the vocabulary allowed for the same intent or intent element, the content template is not applicable as it specifies a choice that is violating another explicit constraint set by a vocabulary template.

A class is an instance of `rdfs:Class`. If allowed vocabulary is a class, this means a conforming intent expression can contain instances of this class. Subclasses of an allowed class are not implicitly allowed as well. This means, it is possible to include instances of an allowed class into an intent expression, but not necessarily instances of a subclass of the allowed class. Instances of a class are only allowed if the class is either explicitly mentioned as allowed vocabulary, or if it is defined as subclass of an allowed class within the intent expression. This means, an intent expression can contain definitions of subclasses of allowed classes and then specify instances of the new class.

Data types also need to be included in the allowed vocabulary to be applicable. If a datatype is included, a literal based on this datatype can be used in a conforming intent expression.

© TM Forum 2024. All Rights Reserved.

A property is an instance of `rdf:Property`. If allowed vocabulary is a property, statements using this property are allowed as long as the subject and object used in its domain and range also conform with rules and constraints of the intent specification. Subproperties of an allowed property are not implicitly allowed as well. This means it is possible to include statements with allowed properties into an assembled intent expression, but not necessarily statements based on subproperties of an allowed property. Statements with subproperties are only allowed if the subproperty is either explicitly mentioned as allowed vocabulary as well, or if it is specified as subproperty of an allowed property within the assembled intent expression.

The TM Forum intent ontology (TIO) defines functions and operators as properties according to RDFS. Also types of metrics are introduced and categorized as properties and subproperties. With respect to vocabulary constraints they would be treated as properties.

For example:

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  insp:availableVocabulary ex:VT1 ;
.
ex:VT1
  a insp:VocabularyTemplate ;
  set:difference ( ex:VocabularySet ex:ExcludedVocabulary )
.
ex:VocabularySet
  a rdfs:Container ;
  set:union ( icm:Vocabulary
              imo:Vocabulary
              imo:VocabularyRDF
              imo:VacabularyRDFS
              imo:VocabularyXMLSchema
              imo:VocabularyTime
              quan:Vocabulary
              xsd:Vocabulary
              set:Vocabulaty
              log:Vocabulary
              fun:Vocabulary
              met:Vocabulary
              iv:Vocabulary
              ex:SliceVocabulary
              [ rdfs:member ex:MyValidityReportingExpectation ]
            )
.
ex:ExcludedVocabulary
  a rdfs:Container ;
  rdfs:member ex:sliceLatencySDO1
  rdfs:member iv:ValidityReportingExpectation
.
```

© TM Forum 2024. All Rights Reserved.

This example defines the vocabulary allowed to be used within an intent. It is defined as difference of two sets. The first argument of the set different function is specified by container `ex:VocabularySet`. It contains vocabulary from multiple distinct models and name spaces in the TM Forum Intent ontology. Additionally, it adds the vocabulary container `ex:SliceVocabulary`. This example assumes a model about network slices and `ex:SliceVocabulary` refers to its vocabulary. This might include slice specific metrics, such as `ex:sliceLatencySD01`. Furthermore, `ex:MyValidityReportingExpectation` is added as member to a container that is included into the union that determines the members of `ex:VocabularySet`.

This example also defines the container `ex:ExcludedVocabulary`. Its members are excluded from the vocabulary template. This means, `ex:sliceLatencySD01` and `iv:ValidityReportingExpectation` are excluded from available vocabulary.

A conforming intent expression might be the following:

```
ex:I2
  a icm:Intent ;
  log:allOf ( ex:E1 ex:E2 )
.
ex:T1
  a icm:Target ;
  rdfs
.
ex:E1
  a icm:PropertyExpectation ;
  icm:target ex:T1 ;
  log:allOf ( ex:C1 )
.
ex:C1
  a log:Condition ;
  quan:smaller ( [ met:lastValue (ex:sliceLatencyTMF) ]
                 "10ms"^^quan:quantity ;
               )
.
ex:ReportEvent1
  a rdfs:Class ;
  rdfs:subClassOf imo:Event ;
  log:match ( [ t:after [ imo:timeOfLastReport ( ex:Intent1 ) ] ;
                t:hasDuration "PT10M"^^xsd:duration
              ]
              t:before
              [ t:hasBeginning imo:Now ]
            ) ;
  imo:eventFor ex:Intent1
.
ex:E2
  a ex:MyValidityReportingExpectation ;
  icm:target ex:T1 ;
  icm:reportTriggers [ a rdfs:Container ;
                       rdfs:member imo:Degrades ;
                       rdfs:member ex:ReportEvent1 ]
.
```

© TM Forum 2024. All Rights Reserved.

This example intent expression uses vocabulary from multiple sets of vocabulary, which are allowed according to the vocabulary template ex:VT1 in the intent specification. For example, the instance `ex:I1` is of type `icm:Intent`. This is allowed, because the class `icm:Intent` is a member of the vocabulary container `icm:Vocabulary`. The property "a" is an abbreviation of "`rdf:type`". This is from the vocabulary of RDF/RDFS.

According to the vocabulary template `ex:VT1`, the class `iv:ValidityReportingExpectation` is not available, but the class `ex:MyValidityReportingExpectation` is. The example intent expression therefore uses `ex:MyValidityReportingExpectation`.

The class `ex:ReportEvent1` is not included in any allowed vocabulary. It is defined in the intent expression. It is however defined as subclass of class `imo:Event`, which is allowed vocabulary and defining a subclass of an allowed class is allowed.

The condition `ex:C1` uses the metric `ex:sliceLatencyTMF`. It avoids using the metric `ex:sliceLatencySDO1`, which was excluded from allowed vocabulary.

7.2. Generic content from vocabulary templates

Vocabulary templates represent constraints regarding allowed vocabulary. However, intent expressions conforming to an intent specifications are based on adding content that is either explicitly prescribed or included according to content templates. Within an intent specification, content templates steer the application of flexible content to a conforming intent expression. Object specifications contribute by allowing a choice of values. This can be done by prescribing statements and values more or less rigidly as described in chapters 4 and 5. With vocabulary templates it is possible to allow an intent owner to define any content that conforms with a set of vocabulary:

The property `insp:contentUsing` represents content that is expressed in accordance with vocabulary templates. If it is used as property of a content template and represents a choice regarding content to be used in an intent expression made by the process that assembles the intent. This content is chosen in accordance with available vocabulary according to the vocabulary template in its range. This can, for example, be an intent manager. Any content that meets the vocabulary constraints set by the function arguments is applicable.

The domain of `insp:contentUsing` is `insp:ContentTemplate`. The content chosen according to vocabulary is therefore applicable to intent expressions through statements with the properties `insp:option`, `insp:alternative` or `insp:mandatory`. It therefore takes a similar role as `insp.content`. While `insp:content` prescribes to use all statements from the object in its range, `insp:contentUsing` allows the process that assembles an intent expression to define content freely as long as it conforms to vocabulary constraints.

The range of `insp:contentUsing` is `insp:VocabularyTemplate`. This vocabulary template represents the constraints to be applied when generating the content for the intent expression.

This definition of content templates is very permissive. It typically allows a broad range of content for an intent expression. This can be used with limited scope to determine the content of intent elements such as individual instances of conditions or expectations. In his case the vocabulary might be restricted to a few properties and classes.

© TM Forum 2024. All Rights Reserved.

The generation of content from vocabulary templates can also be used to allow and specify extensive structures in the intent expression. For example, it can allow expressing multiple property expectations and conditions with logical operators available to assign the conditions and multiple metrics available to be used in the conditions.

Example: Intent restricted only by vocabulary.

```
ex:IS1
  a insp:IntentSpecification ;
  insp:allowedIntent [ insp:mandatory ex:CT1 ]
.
ex:CT1
  a insp:ContentTemplate ;
  insp:contentUsing ex:VT1
.
ex:VT1
  a insp:VocabularyTemplate ;
  set:union ( icm:Vocabulary
              imo:Vocabulary
              imo:VocabularyRDF
              imo:VacabularyRDFS
              imo:VocabularyXMLSchema
              imo:VocabularyTime
              quan:Vocabulary
              xsd:Vocabulary
              set:Vocabulaty
              log:Vocabulary
              fun:Vocabulary
              met:Vocabulary
              iv:Vocabulary
              ex:SliceVocabulary
            )
.
```

This example intent specification allows an intent to be generated based on a vocabulary template. The content of the intent is only restricted by the vocabulary template `ex:VT1`. The example states that an allowed intent according to this intent specification is determined by the content specification `ex:CT1`. Its content is any expression that uses only the vocabulary represented by the vocabulary context `ex:VT1`. However, the set of allowed vocabulary in this example is quite extensive. It includes the vocabulary of the TIO base ontology with multiple distinct name spaces, the entire intent common model, the intent validity extension model and a custom extension model regarding slices. An intent specification like this might be used by an intent manager communicate its general support for vocabulary. Using entire sets of vocabulary from distinct constituent models means that this intent specification expresses which models would be included in the model federation for intent expression. A typical use case for this intent specification is an intent manager capability profile.

© TM Forum 2024. All Rights Reserved.

Example: Generic expression of conditions from vocabulary.

```
ex:IS1
  a insp:IntentSpecification ;
  insp:allowedIntent ex:I1
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1 ;
  a icm:Target
.
ex:E1
  a icm:PropertyExpectation ;
  icm:target ex:T1 ;
  log:allOf ( ex:C1 )
.
ex:C1
  a log:Condition ;
  insp:mandatory ex:CT1
.
ex:CT1
  a insp:ContentTemplate ;
  insp:contentUsing ex:VT1
.
ex:VT1
  a insp:VocabularyTemplate ;
  set:union ( ex:ConditionVocabulary
            [ rdfs:member ex:sliceLatencyTMF ,
                          ex:sliceLatency4GPP ,
                          ex:sliceThroughput

            ]
          )
.
ex:ConditionVocabulary
  a rdfs:Container ;
  set:union ( imo:VocabularyRDF
            imo:VocabularyRDFS
            imo:VocabularyXSD
            log:Vocabulary
            met:Vocabulary
            quan:vocabulary )
.
```

This example intent specification defines an allowed intent with a property expectation, target and condition. The condition needs to be expressed with statements according content template `ex:CT1`. This content template allows any content according to the vocabulary template `ex:VT1`. The allowed vocabulary includes all that is generally needed to define conditions. This includes vocabulary from RDF, RDFS and XML

© TM Forum 2024. All Rights Reserved.

Schema such as the vocabulary from the logical operators ontology, the metrics and observation ontology and the quantity ontology. This is contributed to `ex:VT1` through the dedicated container `ex:ConditionVocabulary`. Further vocabulary includes distinct metrics. Here, three slice related metrics are added. This means, the condition can be expressed using any of these three metrics in any combination.

With this type of choice for expressing content of an intent specification it is not possible to use object templates to define set and ranges for allowed objects and values, because the structure of the content is not specified in detail therefore and anchor point for the object template is not available from the intent specification. If this level of specification is needed, a more prescriptive content definition as described in chapter 5 needs to be used.

© TM Forum 2024. All Rights Reserved.

8. Combined use of templates

Within an intent specification the combination of object templates and content templates provides a high degree of expressiveness. This chapter demonstrates this with examples.

8.1. Example: Choice of metrics and required values.

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1
  a icm:Target ;
  rdf:member ex:Slice1
.
ex:E1
  a icm:PropertyExpectation ;
  icm:target ex:T1 ;
  log:allOf ( ex:C1 )
.
ex:C1
  a log:Condition ;
  insp:mandatory ex:CT1 ;
  insp:option ex:CT2
.
ex:CT1
  a insp:ContentTemplate ;
  insp:content [ quan:smaller ( [ met:lastValue ( ex:OT1 ) ]
                                ex:OT2
                              )
               ]
.
ex:OT1
  a insp:ObjectTemplate ;
  insp:templateType met:metric ;
  insp:allowedValues [ rdfs:member ex:sliceLatency3GPP,
ex:sliceLatencyTMF ] ;
.
ex:OT2
  a insp:ObjectTemplate ;
  insp:templateType quan:Quantity ;
  insp:allowedValues [ insp:selectValues ( _:LatencyValue ex:C2 ... ) ]
.
ex:C2
```

© TM Forum 2024. All Rights Reserved.

```
  a log:Condition ;
  quan:inRange ( _:LatencyValue
                "10ms"^^quan:quantity
                "100ms"^^quan:quantity
              )
.
ex:CT2
  a insp:ContentTemplate ;
  insp:content [ quan:greater( [ met:lastValue ( ex:availability ) ]
                             [ insp:templateType xsd:decimal ;
                               insp:allowedValues [ insp:selectValues (
_:AvailabilityValue ex:C3 ... ) ]
                            )
              ]
.
ex:C3
  a log:Condition ;
  quan:smaller ( _:AvailabilityValue
                "99.5%"^^quan:quantity
              )
.
```

This example shows an intent specification that defines expression options using a combination of content templates and object templates. The specification mainly concerns the content of the condition `ex:C1`. Its role is to specify requirement details for the property expectation `ex:E1`. Within `ex:C1` mandatory content regarding a latency requirement is contributed by the content template `ex:CT1`. Additional optional content regarding an availability requirement is possible according to the content template `ex:CT2`.

The mandatory content for `ex:C1` accoding to `ex:CT1` involves a condition statement using a `quan:smaller` function. Its first argument contains the object template `ex:OT1`, which specifies a possible choice of two distinct metrics `ex:sliceLatency3GPP` and `ex:sliceLatencyTMF`. These metric names are chosen for demonstration purposes. The second argument of the smaller operator is determined by the object template `ex:OT2`. It uses the condition ex:C2 to determine possible values. Here values from 10ms up to 100ms are possible to be chosen. This means that a requirement must be added to the intent expression based on a choice between two latency metrics and using required values between 10ms and 100ms.

The optional content for condition `ex:C1` is determined by the content template `ex:CT2`. It concerns the function `quan:greater` with the metric `ex:availability` and another object template using the condition `ex:C3` to specify allowed values to be chosen in the condition for this metric. This example states that chosen values shall be smaller than "99.5%". Please note that this second object template is specified with a B-node rather than providing a distinct name for it. This demonstrates the variety possible expressions for similar purposes in RDF models.

© TM Forum 2024. All Rights Reserved.

The following condition within an intent expression would be conforming to the intent specification regarding the condition `ex:C1`.

```
ex:C4
  a log:Condition ;
  quan:smaller ( [ met:lastValue ( ex:sliceLatencyTMF ) ]
                 "20ms"^^quan:quantity
               ) ;
  quan:greater ( [ met:lastValue ( ex:availability ) ]
                 "99.1%"^^xsd:decimal
               )
.
```

This condition contains the mandatory content regarding slice latency within a `quan:smaller` function. The chosen required value of 20ms is from the allowed value range. This expression of the condition also contains optional content regarding availability within the arguments of a `quan:greater` function. The required value of "99.1%" is allowed according to the intent specification.

© TM Forum 2024. All Rights Reserved.

9. Dependencies between choices

Assembling intent from intent specifications involved making choices regarding conditions and constraints set by templates. An intent specification might need to specify that choices made for one section of the intent depend on choices made for other sections or additional information about the environment the intent is assembled for. The following chapters introduce vocabulary to express theses dependencies within an intent specification.

9.1. Conditions based on contextual information about the environment

The following properties introduce contextual information from the intent owner's processes and in particular from the processes involved in assembling and intent expression:

`insp:chosenHandlingDomain`, `insp:chosenIntentHandler`, `insp:intentHandlerVendor`
An intent owner determines which other autonomous domains it needs to comply to requirements. The intent owner would assemble an intent for the purpose of communicating these requirements to the intent handler within the selected domain. This mean at the time of intent assembly from intent specifications, the intent owner has selected and domain and intent handler already.

The property `insp:chosenHandlingDomain` refers to a chosen target autonomous domain. The intent expression is generated from the intent specification to express the requirements for that autonomous domain.

The property `insp:chosenIntentHandler` refers to the instance of an intent manager within the chosen target autonomous domain. The intent expression is generated from the intent specification to be sent to this intent manager instance.

The property `insp:intentHandlerVendor` refers to the vendor of the intent manager that is chosen to be the handler of the Intent that is generated from the intent specification.

`insp:intentOwner`, `insp:intentOwnerDomain`, `insp:intentOwnerVendor`
The property `insp:intentOwner` refers to the intent manager that is assembling the intent from the intent specification. It will be the intent owner of the intent when this intent is used and sent to an intent handler.

The property `insp:intentOwnerDomain` refers to the autonomous domain of the intent manager that is assembling the intent from the intent specification.

The property `insp:intentOwnerVendor` refers to the vendor of the intent manager that is assembling the intent from the intent specification.

These properties are used similarly to metrics and therefore they are modeled as sub-property of `met:metric`. The intent managers, domains and vendors they identify are therefore observations. Consequently, these properties can be used to express conditions and allow conditional branches within intent specifications. This is demonstrated for conditional templates in the following chapters. The evaluation of conditions is executed when assembling an intent expression to be used for sending intent to an intent handler. The intent owner needs to make sure that the intent conforms to a respective intent specification when it is sending the intent.

© TM Forum 2024. All Rights Reserved.

Further properties can be specified by additional models as needed.

9.2. Conditions based on other content choices

A section of options or alternatives for content templates or choices made in object templates might impact other choices while assembling an intent. For example, if an option is taken another option in a different part of the intent and based on a different template might not be eligible anymore. The functions introduced in this chapter enable to specify conditions within an intent specification that establish a dependency on choices made.

A choice regarding content templates is represented by the condition operators `insp:chosenAny`, `insp:chosenAll`, `insp:chosenAnyFor`, or `insp:chosenAllFor` These operators are modeled as functions according to the function definition ontology [TR292C]. They have boolean type result values and can therefore be used to express conditions.

`<xsd:boolean> = insp:chosenAll ( <insp:ContentTemplate> ... )`The operator `insp:chosenAll` indicates true, if all content templates referred to by its arguments were chosen.

`<xsd:boolean> = insp:chosenAny ( <insp:ContentTemplate> ... )`The operator `insp:chosenAny` indicates true, if any of the content templates referred to by its arguments were chosen.

Typically the intent specification uses content templates to specify what detailed content must or can be used for an intent element. It is possible that the same template species choices for multiple distinct intent elements. The operators `insp:chosenAny` and `insp:chosenAll` do not distinguish what intent element instance the choice was made for. If this distinction is relevant, the operators `insp:chosenAnyFor`, or `insp:chosenAllFor` can be used instead. They represent individual choices distinguished by intent element instance in combination with chosen templates. Their first argument is an intent element used in the intent specification for which a choice based on a content template might have been made.

`<xsd:boolean> = insp:chosenAllFor ( <intent element> <insp:ContentTemplate> ... )`The operator `insp:chosenAllFor` indicates true, if for the intent element instance given as first argument all content templates referred to by its second and further arguments were chosen.

`<xsd:boolean> = insp:chosenAnyFor ( <intent element> <insp:ContentTemplate> ... )`
The operator `insp:chosenAnyFor` indicates true, if for the intent element instance given as first argument any of content templates referred to by its second and further arguments were chosen.

Examples for the use of these operator functions are provided later sub-chapters.

9.3. Conditions based on chosen objects and values

Object templates allow to select objects and values to be used. Other choices in the intent assembly process might depend on the sections made for an object template.

`<xsd:boolean> = insp:valueSelected ( <insp:ObjectTemplate> <rdfs:Container> ... )`
The operator `insp:valueSelected` indicates true, if for the object template referred to

© TM Forum 2024. All Rights Reserved.

by the first argument, the chosen value is a member of any of the containers provided as second or remaining arguments.

An individual object template can determine multiple distinct choices at various locations within the assembled intent. The operator `insp:valueSelected` would refer to any of these distinct choices. It does not distinguish the intent element the choice is made for. This is considered however in the operator `insp:valueSelectedFor`.

```
<xsd:boolean> = insp:valueSelectedFor ( <intent element>
<insp:ObjectTemplate> <rdfs:Container> ... )
```

The operator `insp:valueSelectedFor` indicates true, if certain values were chosen for the combination of an intent element and object template. The intent element to be considered is provided by the first argument and the object template is referred to by the second argument. The values to be checked are members of any of the containers provided by the third or remaining arguments.

In these operator functions, the values to be checked are members of containers. These containers can be determined by enumerating individual members using the `rdfs:member` property. Additionally, functions for container construction and member filtering can be used. Such functions are defined, for example, in the set operators ontology [TR292F]. Also the `insp:selectValue` function can be used to determine container members based on a condition. This is particularly useful for expressing value ranges.

Examples for the use of these operator functions are provided later sub-chapters.

9.4. Conditions based on used vocabulary

Vocabulary templates define sets of intent expression vocabulary that can be used for expressing intent or parts of an intent. Choices made in the intent expression can depend on vocabulary used.

```
<xsd:boolean> = insp:usedVocabularyFor ( <intent element>
<rdfs:Container> ... )
```

The function `insp:usedVocabularyFor` indicates "true", if for expressing the intent element referred to by the first argument all vocabulary is used specified in any of the remaining arguments. The second and remaining arguments are containers with members that refer to distinct elements of intent expression vocabulary.

Examples for the use of these operator functions are provided later sub-chapters.

9.5. Conditionally applicable content

Content templates can state a condition for them to be applicable. The properties `insp:applicableIf` and `insp:applicableIfNot` assign a condition to an content template. Their range is therefore `log:Condition`. The property `insp:applicableIf` specifies that the content template in its domain is applicable to the intent expression if the condition in its range evaluates to true. Respectively, the property `insp:applicableIfNot` specifies that the content template in its domain is applicable to the intent expression if the condition in its range evaluates to false.

A template being applicable does not enforce that its content must be included in the intent. It rather states that the template is available to be chosen and used in the intent expression. If a content template is not applicable, it is not an option or a viable alternative and cannot be mandatory. This means, if a content template used as option

© TM Forum 2024. All Rights Reserved.

is not applicable, this option cannot be chosen. If it is used as alternative and not applicable, this alternative cannot be chosen and other alternatives must be used. If no viable alternative is left, because they all have conditions that makes then not applicable, no content is contributed from the respective alternative statement. Similarly, if the conditions indicate that the template used by a mandatory statement is not applicable, this mandatory statement has no content to contribute. This means, even mandatory content is skipped if it is not applicable according to its template conditions.

Multiple conditions given with `insp:applicableIf` and `insp:applicableIfNot` within a single content template are in a conjunction. This means the content template is only applicable if all conditions confirm it. If conditions shall have a different effect on the availability of the content template, logical operators according to TR292E can be used.

Example: Conditional applicability of templates

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1
  a icm:Target ;
  rdf:member ex:Slice1
.
ex:E1
  a icm:PropertyExpectation ;
  icm:target ex:T1 ;
  log:allOf ( ex:C1 )
.
ex:C1
  a log:Condition ;
  insp:alternative ( ex:CT1 ex:CT2 );
.
ex:CT1
  a insp:ContentTemplate ;
  insp:content [ quan:smaller ( [ met:lastValue ( ex:sliceLatencyVENDORX ) ]
                                ex:OT1
                              )
               ] ;
  insp:applicableIf ex:CTC1
.
ex:CTC1
  a log:Condition ;
  log:equal ( [ met:lastValue ( insp:intentHandlerVendor ) ]
               ex:VendorX
              )
.
ex:OT1
```

© TM Forum 2024. All Rights Reserved.

```
  a insp:ObjectTemplate ;
  insp:templateType quan:Quantity ;
  insp:valueCondition ex:VC1 ;
  insp:valueProxy _:LatencyValue
.
ex:VC1
  a log:Condition ;
  quan:inRange ( _:LatencyValue
                 "10ms"^^quan:quantity
                 "100ms"^^quan:quantity
               )
.
ex:CT2
  a insp:ContentTemplate ;
  insp:content [ quan:smaller ( [ met:lastValue ( ex:sliceLatency ) ]
                                ex:OT1
                              )
               ]
.
```

This example intent specification defines alternative content for the condition `ex:C1` whin an viable intent expression. The two alternatives differ in the metric being used. The first alternative, represented by the content template `ex:CT1` allows the metric `ex:sliceLatencyVENDORX`. This alternative is conditionally available. It can only be chosen if the vendor of the intent handler is `ex:VendorX`, expressed by condition `ex:SC1`. The other alternative is not conditional. This means if the intent owner assembles the intent to be sent to an intent handler from Vendor X, both alternatives are available. For other vendors, only the second alternative is available. The range of values to choose from for completing the condition expression of `ex:C1` is determined by the object template `ex:OT1` in both alternatives.

9.5.1. Mutual dependencies

The content choice operators introduced in this chapter 7.2 can be used in content template conditions to express the availability of a content choice based on other choices made in the assembly of an intent expression. This allows to express circular relationships through template conditions where two choices depend on each other. For example, a condition regarding content template X might state that template X is only available to be chosen if template Y is chosen. Another condition regarding template Y might state that template Y is only available if template X is chosen. This logically means that none of them is available unless both are chosen or in other words, the intent owner cannot choose one without also choosing the other.

In another example the conditions might state that template X is only available if template Y is chosen and template Y is not available if template X is chosen. This logically means template X can never be chosen as doing so would always create a contradiction. This means it is possible using conditions to create intent specifications where certain elements of an intent specification are not accessible in any situation. The developer of an intent specification should take this into consideration when expressing conditional templates.

© TM Forum 2024. All Rights Reserved.

9.6. Conditionally available object values

Object and value selection according to object templates might depend on choices made for other parts of the intent or information from the environment of the intent owner. This dependency can be captured by value conditions using the properties and functions introduced in chapters 8.1 through 8.4.

Example: Conditional applicability of templates

```
ex:IS1
  a insp:IntentSpecification ;
  insp:mandatoryIntent ex:I1 ;
.
ex:I1
  a icm:Intent ;
  log:allOf ( ex:E1 )
.
ex:T1
  a icm:Target ;
  rdf:member ex:Slice1
.
ex:E1
  a icm:PropertyExpectation ;
  icm:target ex:T1 ;
  log:allOf ( ex:C1 )
.
ex:C1
  a log:Condition ;
  insp:alternative ( ex:CT1 ex:CT2 );
.
ex:CT1
  a insp:ContentTemplate ;
  insp:content [ quan:smaller ( [ met:lastValue ( ex:sliceLatencyVENDORX ) ]
                                ex:OT1
                              )
               ] ;
  insp:applicableIf ex:CTC1
.
ex:CTC1
  a log:Condition ;
  log:allOf (  log:equal ( [ met:lastValue ( insp:intentHandlerVendor ) ]
                           ex:VendorX
                         )
            )
.
ex:OT1
  a insp:ObjectTemplate ;
  insp:templateType quan:Quantity ;
  insp:allowedValues [ insp:selectValues ( _:value ex:VC1 ... ) ]
  insp:allowedValues [ insp:selectValues ( _:value ex:VC2 ... ) ]
.
ex:VC1
```

© TM Forum 2024. All Rights Reserved.

```
  a log:Condition ;
  insp:chosenAllFor ( ex:C1 ex:CT1 )
  quan:inRange ( _:LatencyValue
                "5ms"^^quan:quantity
                "200ms"^^quan:quantity
              )
.
ex:VC2
  a log:Condition ;
  insp:chosenAllFor ( ex:C1 ex:CT2 )
  quan:inRange ( _:LatencyValue
                "10ms"^^quan:quantity
                "100ms"^^quan:quantity
              )
.
ex:CT2
  a insp:ContentTemplate ;
  insp:content [ quan:smaller ( [ met:lastValue ( ex:sliceLatency ) ]
                              ex:OT1
                            )
             ]
.
```

This example intent specification demonstrates how value choices with and object template can depend on content choices with content templates. The intent expression content for the condition `ex:C1` is determined either by content template `ex:CT1` or by content template `ex:CT2`. The availability of `ex:CT1` depends on the vendor of the intent handler. Consequently, if it is not Vendor X only the alternative represented by `ex:CT2` is available.

The choice of values to be used in `ex:CT1` for expressing a requirement condition is determined by the object template `ex:OT1` and its value conditions `ex:VC1` and `ex:VC2`. This example demonstrates the use of the `insp:chosenAllFor` within an value condition. Values that can be chosen according to `ex:VC1` need to meet to distinct conditions. They need to be in range between 5ms and 200ms and the content according to content template `ex:CT1` was chosen. According to the other value condition `ex:VC2`, values need to be in range between 10ms and 100ms. This means, that values between 5ms and 200ms are only available for the metric `ex:sliceLatencyVENDORX`. Otherwise the choice of values is any value between 10ms and 100ms.

© TM Forum 2024. All Rights Reserved.

10. Generating end verifying intent expressions

The following steps describe how a conforming intent expression can be generated from an intent specification.

- Be aware of the use case and environment of the intent!
  An intent is typically generated by an intent manager to be sent to an autonomous domain and the intent handler within. The selection of the targeted autonomous domain primarily depends on the nodes, services and resources an intent manager needs to impact with requirements. This means an intent manager needs to select a suitable autonomous domain and intent handler. This creates an environment for the generation of an intent. Information about the selected intent handler need to be considered, for example the vendor it originated from. Further relevant information are the exposed services and resources and knowledge about metrics that can be observed.
- Select the relevant intent specification!
  Intent specifications can be provided from various sources. For example, they can be allocated in a product and service catalog and associated with product and service specifications. If an intent shall be generated to express product or service requirements, the intent specification associated with the respective product or service specification would apply. Intent specifications can also be associated with the capabilities of an intent manager. If an intent manager is planned to be used as handler of the intent, this intent specification would apply. It is also possible that multiple intent specifications need to be considered. Furthermore, an intent specification can refer to further intent specification to be applied. An eligible intent expression would need to conform with all applicable intent specifications.
- Use the intents specified in the intent specification!
  Intent specifications are based on intent expressions according to the TM Forum Intent Ontology (TIO). This can include intent extension models and other constituent models within an applicable model federation. Based on this, an intent specification might use intents to explicitly prescribe at least a partial structure and content to be used for an conforming intent expression. The starting point for creating an intent expression in accordance with the intent specification would therefore be the prescribed intents according to `insp:manadtoryIntent` and `insp:availableIntent` properties.
- Choose unique names for instances!
  Naming for instances of intent elements can be chosen as needed when generating an intent expression from an intent specification. It is recommended to choose unique instance names different from the ones used within the intent specification.
- Choose according to content templates and their conditions!
  Content templates define optional, mandatory or alternative statements to be included in an intent expression or not. The applicability of content templates and the statements they contribute can depend on conditions. When an intent is assembled, these conditions need to be evaluated to determine what can be chosen to be included.

© TM Forum 2024. All Rights Reserved.

- Choose according to object templates!
  Object templates in intent specifications describe eligible values and objects to be used in conforming intent expressions. When assembling intent this typically determines the finer details of a conforming intent expression. Choose values accordingly to furmulate requirement details in the intent expression!

If the task is to assemble an intent, the steps described above deliver an intent expression that meets all the rules and constraints of an intent specification. If the task is to verify if an intent is expressed in accordance with an intent specification these steps indicate what to check.

© TM Forum 2024. All Rights Reserved.

11.Administrative Appendix

11.1. Document History

11.1.1. Version History

Version Number	Date Modified	Modified by:	Description of changes
3.5.0	03-May-2024	Alan Pope	Final edits prior to publication
3.6.0	04-Jul-2024	Alan Pope	Final edits prior to publication

11.1.2. Release History

Release Status	Date Modified	Modified by:	Description of changes
Pre-production	03-May-2024	Alan Pope	Initial release
Production	28-Jun-2024	Adrienne Walcott	Updated to reflect TM Forum Approved status
Pre-production	04-Jul-2024	Alan Pope	Updated to v3.6.0
Production	30-Aug-2024	Adrienne Walcott	Updated to reflect TM Forum Approved status

11.2. Acknowledgments

Team Member (@mention)	Company	Role*
Jörg Niemöller	Ericsson	Author, Project Co-Chair
Kevin McDonnell	Huawei	Project Co-Chair
Yuval Stein	Amdocs	Project Co-Chair
Kamal Maghsoudlou	Ericsson	Key Contributor
Leonid Mokrushin	Ericsson	Key Contributor
Marin Orlić	Ericsson	Key Contributor
Alan Pope	TM Forum	Reviewer, Editor
Dave Milham	TM Forum	Additional Input
Xiao Hongmei	Inspur	Reviewer

**Select from: Project Chair, Project Co-Chair, Author, Editor, Key Contributor, Additional Input, Reviewer*

© TM Forum 2024. All Rights Reserved.

12. TR299 Intent Specification v3.6.0 - Appendix A Vocabulary

12.1. Appendix A: Vocabulary Reference

This chapter contains a reference definition of all model vocabulary. It is sorted alphabetically.

12.1.1. allowedValues

The property `insp:allowedValues` is used to specify the set of candidate objects or values to choose from for substituting the object template in its domain within a conforming intent expression.

Instance of: `rdf:Property`Domain: `insp:ObjectTemplate`Range: rdfs:Container

12.1.2. alternative

The property `insp:alternative` specifies alternative content within intent specifications. Its range contains a list of all content templates representing viable alternatives. Each content template in the list alternative represents a statement or set of statements. Exactly one of these content templates and its statements needs to be used in place of `insp:alternative` when expressing an intent. This means a conforming intent expression must include a set of statements for every `inst:alternative` statement in the intent specification.

Instance of: `rdf:Property`Range: `insp:ContentTemplate`

12.1.3. applicableIf

The property `insp:applicableIf` specifies that the content template in its domain is applicable to the intent expression if the condition in its range evaluates to true.

Instance of: `rdf:Property`Domain: `insp:ContentTemplate`Range: `log:Condition`

12.1.4. applicableIfNot

The property `insp:applicableIfNot` specifies that the content template in its domain is applicable to the intent expression if the condition in its range evaluates to false.

Instance of: `rdf:Property`Domain: `insp:ContentTemplate`Range: `log:Condition`

12.1.5. availableVocabulary

The property `insp:availableVocabulary` assigns a vocabulary template to an intent within an intent specification. It implies that the intent and all its elements can only be expressed using the vocabulary specified by the vocabulary template. Multiple `insp:availableVocabulary` assigned to the same subject are interpreted as union of the individual vocabulary templates.

Instance of: `rdf:Property`Range: `insp:VocabularyTemplate`

© TM Forum 2024. All Rights Reserved.

12.1.6. chosenAll

The operator `insp:chosenAll` indicates true, if all content templates referred to by its arguments were chosen.

```
<xsd:boolean> = insp:chosenAll ( <insp:ContentTemplate> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 1
Argument types: All arguments have type `insp:ContentTemplate`.

12.1.7. chosenAllFor

The operator `insp:chosenAllFor` indicates true, if for the intent element instance given as first argument all content templates referred to by its second and further arguments were chosen.

```
<xsd:boolean> = insp:chosenAllFor ( <intent element>
<insp:ContentTemplate> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 2
Argument types: The first argument is of type `rdfs:Resource`. It is referring to an element of the intent. The second and remaining arguments have type `insp:ContentTemplate`.

12.1.8. chosenAny

The operator `insp:chosenAny` indicates true, if any of the content templates referred to by its arguments were chosen.

```
<xsd:boolean> = insp:chosenAny ( <insp:ContentTemplate> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 1
Argument types: All arguments have type `insp:ContentTemplate`.

12.1.9. chosenAnyFor

The operator `insp:chosenAnyFor` indicates true, if for the intent element instance given as first argument any of content templates referred to by its second and further arguments were chosen.

```
<xsd:boolean> = insp:chosenAnyFor ( <intent element>
<insp:ContentTemplate> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 2
Argument types: The first argument is of type `rdfs:Resource`. It is referring to an element of the intent. The second and remaining arguments have type `insp:ContentTemplate`.

© TM Forum 2024. All Rights Reserved.

12.1.10. chosenHandlingDomain

The property `insp:chosenHandlingDomain` refers to a chosen target autonomous domain. The intent expression is generated from the intent specification to express the requirements for that autonomous domain.

Instance of: `rdf:Property`Subproperty: `met:metric`

12.1.11. chosenIntentHandler

The property `insp:chosenIntentHandler` refers to the instance of an intent manager within the chosen target autonomous domain. The intent expression is generated from the intent specification to be sent to this intent manager instance.

Instance of: `rdf:Property`Subproperty: `met:metric`

12.1.12. content

The property `insp:content` assigns the statements represented by the content template. The node used in its range is the source of statements to be transferred into the intent expression. All statements in that node shall be transferred. If `insp:content` is used multiple times within a content template, their contributions are additive and the superset of statements is applied.

Instance of: `rdf:Property`Domain: `insp:ContentTemplate`

12.1.13. contentUsing

The property `insp:contentUsing` represents content that is expressed in accordance with vocabulary templates. If it is used as property of a content template and represents a choice regarding content to be used in an intent expression made by the process that assembles the intent. This content is chosen in accordance with available vocabulary according to the vocabulary template in its range.

Instance of: `rdf:Property`Domain: `insp:ContentTemplate`

12.1.14. ContentTemplate

Content templates are instances of class `insp:ContentTemplate`, which is a subclass of `insp:Template`.

Instance of: `rdfs:Class`Subclass of: `insp:Template`

12.1.15. contentUsing

The property `insp:contentUsing` assigns content chosen according to a vocabulary template.

Instance of: `rdf:Property`Domain: `insp:ContentTemplate`Range: `insp:VocabularyTemplate`

12.1.16. intentHandlerVendor

The property `insp:intentHandlerVendor` refers to the vendor of the intent manager that is chosen to be the handler of the Intent that is generated from the intent specification.

Instance of: `rdf:Property`Subproperty: `met:metric`

© TM Forum 2024. All Rights Reserved.

12.1.17. intentOwner

The property `insp:intentOwner` refers to the intent manager that is assembling the intent from the intent specification.

Instance of: `rdf:Property`Subproperty: `met:metric`

12.1.18. intentOwnerDomain

The property `insp:intentOwnerDomain` refers to the autonomous domain of the intent manager that is assembling the intent from the intent specification.

Instance of: `rdf:Property`Subproperty: `met:metric`

12.1.19. intentOwnerVendor

The property `insp:intentOwnerVendor` refers to the vendor of the intent manager that is assembling the intent from the intent specification.

Instance of: `rdf:Property`Subproperty: `met:metric`

12.1.20. IntentSpecification

Intent specifications are instances of class `insp:IntentSpecification`. An intent is assembled in accordance to an intent specification by finding an intent expression that complies to all rules and constraints expressed by the intent specification.

Instance of: `rdfs:Class`

12.1.21. mandatory

The property `insp:mandatory` specifies a content template that represents a statement or sets of statements that must be included in an intent expression. Its range contains a content template instance of class `insp:ContentTemplate`. This content template determines the statements that must be used as substitute for the `insp:mandatory` statement in a conforming intent expression.

Instance of: `rdf:Property`Range: `insp:ContentTemplate`

12.1.22. mandatoryIntent

The property `insp:mandatoryIntent` intent assigns intent expressions that must be used. This means an intent expression conforming to the intent specification must conform to this intent expression including all rules and dynamic content choices it allows.

Instance of: `rdf:Property`Domain: `insp:IntentSpecification`Range: `icm:Intent`

12.1.23. mandatorySpecification

The property `insp:mandatorySpecification` expresses that an eligible intent expression must conform to the intent specification provided in its range. This expresses that the rules and constraints of that intent specification are mandatory for follow. Multiple statements using `insp:mandatorySpecification` with the same subject are in a conjunction (AND). This means an assembled intent expression must conform to all mandatory intent specifications.

Instance of: `rdf:Property`Domain: `insp:IntentSpecification`Range: `insp:IntentSpecification`

© TM Forum 2024. All Rights Reserved.

12.1.24. ObjectTemplate

An object template is an instance of class `insp:ObjectTemplate`, which is a subclass of `insp:Template`. Within intent specifications an object template can be used as substitute for any object within an intent expression.

Instance of: `rdfs:Class`Subclass of: `insp:Template`

12.1.25. option

The property `insp:option` specifies a set of statements that can be used in an intent expression. The range of `insp:option` is a template of class `insp:ContentTemplate`. The statements and expressions of this content template would substitute the insp:option statement within a conforming intent expression if this option is chosen.

Instance of: `rdf:Property`Range: `insp:ContentTemplate`

12.1.26. optionalIntent

The property `insp:optionalIntent` assigns intent objects to an intent specification. An intent expression that resembles any of these intents conforms to the intent specification.

Instance of: `rdf:Property`Domain: `insp:IntentSpecification`Range: `icm:Intent`

12.1.27. optionalSpecification

The property `insp:optionalSpecification` expresses that an intent expression can be assembled following the intent specifications provided in its range. Multiple statements using `insp:optionalSpecification` with the same subject are in a disjunction (OR). This means an intent expression conforms overall if it conforms to at least one and potentially several of the optional specifications.

Instance of: `rdf:Property`Domain: `insp:IntentSpecification`Range: `insp:IntentSpecification`

12.1.28. selectValues

The abstract function `insp:selectValues` represents a container of values that can be chosen to substitute an object template. The first argument of `insp:selectValues` is a variable that represents the values to be chosen. The second and further arguments are conditions that use the variable in its first argument to express condition statements. Every value possible for the variable in the first argument according to the conditions is a member of the resulting container. The conditions are in a conjunction. This means, a value needs to satisfy all conditions to be included in the result container

```
<rdfs:Container> = insp:selectValues ( <rdfs:Resource> <log:Condition>
... )
```

Instance of: `fun:Function`
Result type: `rdfs:Container`
Arity: At least 2
Argument types: The first argument is any `rdfs:Resource`. The second and further arguments are `log:Condition`

© TM Forum 2024. All Rights Reserved.

12.1.29. Template

Templates are instances of subclasses of class `insp:Template`.

Instance of: `rdfs:Class`

12.1.30. templateType

The property `insp:templateType` specifies what type of object an object template instance is representing and substituting in an intent expression that conforms to the intent specification.

Instance of: `rdf:Property`Domain: `insp:ObjectTemplate`

12.1.31. usedVocabularyFor

The function `insp:usedVocabularyFor` indicates "true", if for expressing the intent element referred to by the first argument all vocabulary is used specified in any of the remaining arguments. The second and remaining arguments are containers with members that refer to distinct elements of intent expression vocabulary.

```
<xsd:boolean> = insp:usedVocabularyFor ( <intent element>
<rdfs:Container> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 2
Argument types: The first argument is of type `rdfs:Resource`. It is referring to an element of the intent. The second and remaining arguments have type `rdfs:Container`. They define the set of vocabulary the function is checking against.

12.1.32. valueSelected

The operator `insp:valueSelected` indicates true, if for the object template referred to by the first argument, the chosen value is a member of the containers provided as second or remaining arguments.

```
<xsd:boolean> = insp:valueSelected ( <insp:ObjectTemplate>
<rdfs:Container> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 2
Argument types: The first argument is of type `insp:ObjectTemplate`. The second and remaining arguments have type `insp:ContentTemplate`.

12.1.33. valueSelectedFor

The operator `insp:valueSelectedFor` indicates true, if certain values were chosen for the combination of an intent element and object template. The intent element to be considered is provided by the first argument and the object template is referred to by the second argument. The values to be checked are members of the containers provided by the third or remaining arguments.

```
<xsd:boolean> = insp:valueSelectedFor ( <intent
element>                                        <insp:ObjectTemplate>
                                        <rdfs:Container> ... )
```

Instance of: `fun:Function`
Result type: `xsd:boolean`
Arity: At least 3

© TM Forum 2024. All Rights Reserved.

Argument types: The first argument is of type `rdfs:Resource`. It is referring to an element of the intent. The second argument is of type `insp:ObjectTemplate`. The third and remaining arguments have type `insp:ContentTemplate`.

12.1.34. Vocabulary

The object `insp:Vocabulary` is a container of all model elements.

Instance of: `rdfs:Container`

12.1.35. VocabularyTemplate

A vocabulary template is an instance of class `insp:VocabularyTemplate`, which is a subclass of `insp:Template`. A vocabulary template is also a subclass of `imo:Vocabulary`, which is a subclass of `rdfs:Container`.

Instance of: `rdfs:Class`Subclass of: `insp:Template`Subclass of: `imo:Vocabulary`

© TM Forum 2024. All Rights Reserved.